KU LEUVEN

Labo 4 - CUDA

Sander De Stercke (MEIECFS)
Isaak Vervenne (MEISWE)

Maart 2026

1 Wat is er gerealiseerd?

Dit labo legt de nadruk op de verschillende types geheugen die je kan gebruiken tijdens het GPU programmeren. Als eerste wordt het fenomeen "Coalesced memory access"grondig bestudeerd. We benaderen dit onderwerp aan de hand van snelheidsmetingen van zowel een copy als een transpositiekernel. De metrieken die als basis staan van de studie zijn: effectieve bandbreedte per kernel, executietijd, en performantieverandering bij het schalen van de matrixgrootte en toegangspatronen.

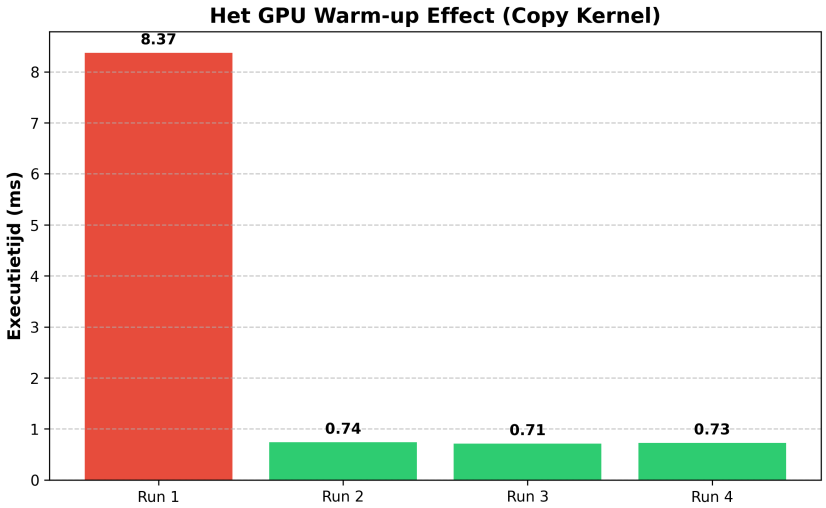
Daarna worden het globale, gedeelde en constante geheugen onder de loep genomen. De kernel die we hierbij gebruiken zal twee matrices vermenigvuldigen, waarbij de executietijd voor elk geheugentype wordt geëvalueerd.

2 Problemen

2.1 Het GPU Warm-up Effect

Tijdens het verzamelen van de meetresultaten voor de copy en transpositiekernels merkten we een sterke afwijking op bij de allereerste uitvoering van het programma. De copy kernel nam tijdens de eerste run ongeveer 8.37 ms in beslag, terwijl dit bij latere runs direct terugviel naar een stabiele waarde.

Dit fenomeen staat ook wel bekend als het "GPU warm-up effect". Wanneer een CUDA-applicatie voor de allereerste keer een kernel lanceert, moet de GPU zijn interne context nog initialiseren (zoals het opzetten van page tables en het initialiseren van de PCIe-bus). Deze opstarttijd wordt bij de eerste meting opgeteld, wat leidt tot sterk vertekende resultaten.



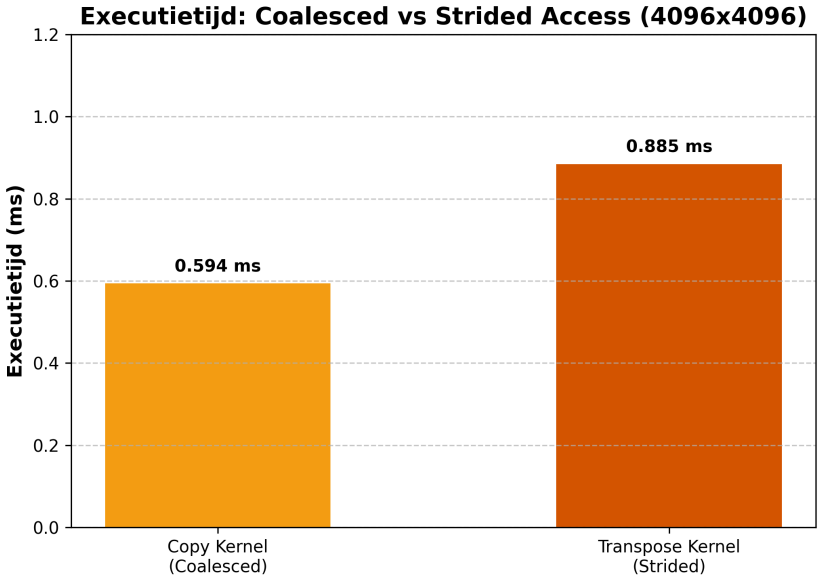
Figuur 1: Illustratie van het GPU warm-up effect: de eerste run is aanzienlijk trager dan de daaropvolgende runs.

Om dit probleem op te lossen en correcte data te verzamelen voor onze analyse, hebben we de effecten van deze initiële overhead verwijderd door warm-up kernels uit te voeren vooraleer we de timers startten.

3 Vragen en Analyse

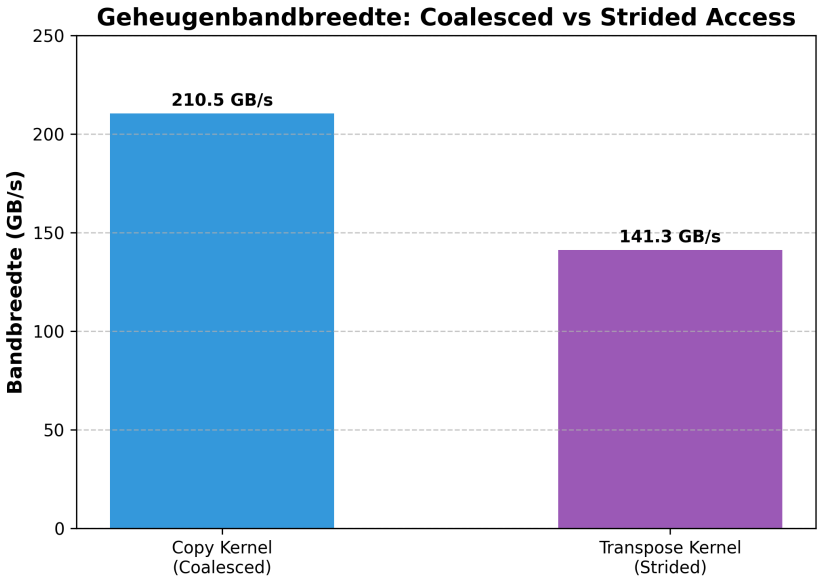
3.1 Deel 1: Coalesced vs. Strided Memory Access

Nadat het warm-up effect werd verwijderd, zagen we resultaten die overeenkomen met de theorie omtrent geheugentoegang op een GPU. Om de impact van geheugentoegangspatronen te aan te tonen, hebben we zowel de executietijd als de effectieve geheugenbandbreedte vergeleken tussen de copy en de transpositiekernel.



Figuur 2: Vergelijking van de executietijd tussen een coalesced (Copy) en strided (Transpositie) toegangspatroon op een 4096×4096 matrix.

Zoals te zien in Figuur 2, wordt de kopie aanzienlijk sneller uitgevoerd (gemiddelde van 0.594 ms) dan de transpositie-operatie (gemiddelde van 0.885 ms). Dit heeft een direct effect op de effectieve bandbreedte die bereikt wordt, te zien in Figuur 3.



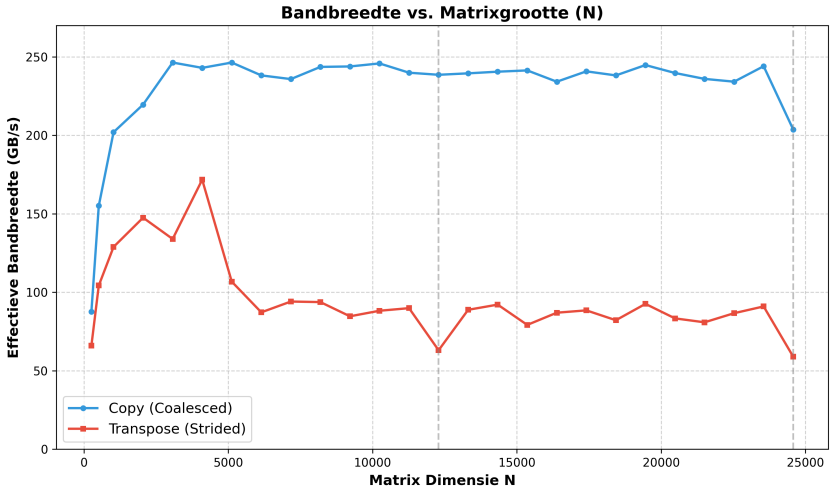
Figuur 3: Vergelijking van de effectieve geheugenbandbreedte. De Copy Kernel behaalt met ∼210.5 GB/s een aanzienlijk hogere doorvoer dan de Transpose Kernel (∼141.3 GB/s).

Het verschil in geheugenbandbreedte komt door **coalesced memory access**. Coalescing treedt op wanneer alle threads binnen een warp opeenvolgende globale geheugenlocaties aanspreken. De GPU kan deze requests dan efficiënt combineren (*coalescen*) in zo min mogelijk geheugentransacties. Zo wordt de data als burst geleverd, wat latency vermindert en dus bandbreedte verhoogt.

Bij het kopiëren van de matrix lezen en schrijven de threads naar opeenvolgende geheugenadressen (coalesced). Bij het transponeren leest de GPU de data nog steeds opeenvolgend in, maar wordt deze weggeschreven op locaties die telkens N elementen uit elkaar liggen (*strided access*). Hierdoor breekt de coalescing bij het wegschrijven, wat resulteert in veel meer (en tragere) geheugentransacties.

3.2 Deel 1b: Schaalbaarheid en Access Patterns

Door het variëren van de matrixgrootte ($N \times N$) in combinatie met verschillende toegangspatronen bekeken we de prestaties van de kernels onder verschillende omstandigheden.



Figuur 4: De effectieve bandbreedte voor de Copy en Transpose kernels naarmate de matrixgrootte N toeneemt. Opvallend zijn de drastische terugvallen van de Transpose kernel bij zeer grote machten van twee, zoals $N = 12288$ en $N = 24576$.

De bandbreedte neemt voor de *Copy Kernel* asymptotisch toe tot deze afvlakt rond de 246 GB/s. Dit komt neer op de realistische maximale bandbreedte van de gebruikte GPU (GTX 1660 Super). Omdat de threads in een warp coalesced lezen en schrijven, wordt de geheugenbus optimaal benut.

De *Transpose Kernel* is daarentegen trager, omdat het *strided* wegschrijven niet coalesced gebeurt. We zien dat bij specifieke dimensies zoals $N = 12288$ en $N = 24576$, de bandbreedte daalt tot amper $\sim$60 GB/s.

Dit kan verklaard worden door **memory partition camping** (soms ook *cache thrashing* genoemd). Het globale geheugen van een GPU zoals de NVIDIA GeForce GTX 1660 Super is fysiek verdeeld over meerdere *memory partitions*. Het partitie-nummer wordt bepaald door de lagere bits van het geheugenadres.

Bij een naïeve transpose schrijven de threads met een *stride* van:

$$\text{stride} = N \cdot \text{sizeof(float)}$$

Aangezien sizeof(float) = 4 bytes geldt:

$$\text{stride} = 4N$$

Voor de problematische dimensies krijgen we dus:

$$N = 12288 \implies \text{stride} = 4 \times 12288 = 49152 \text{ bytes}$$
$$N = 24576 \implies \text{stride} = 4 \times 24576 = 98304 \text{ bytes}$$

Deze waarden zijn grote veelvouden van machten van twee:

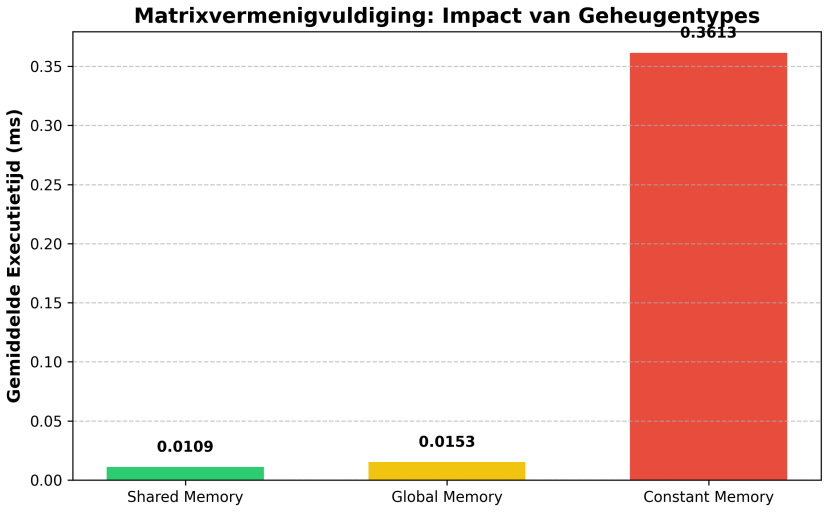
$$12288 = 3 \cdot 2^{12}$$
$$24576 = 6 \cdot 2^{12}$$

Daardoor worden bij het berekenen van het geheugenadres telkens dezelfde lagere adresbits gebruikt voor opeenvolgende threads in een warp. Omdat deze bits bepalen naar welke *memory partition* een adres wordt gestuurd, kan het gebeuren dat een groot aantal geheugenoperaties naar dezelfde partitie wordt gemapt.

Het gevolg is dat veel threads uit dezelfde warp tegelijkertijd naar dezelfde geheugenpartitie proberen te schrijven, terwijl andere partities nauwelijks gebruikt worden. Hierdoor raken bepaalde partities overbelast en moeten de geheugenoperaties geserialiseerd worden. Dit fenomeen staat bekend als *partition camping* en leidt tot een sterke daling van de effectieve geheugenbandbreedte, zoals zichtbaar in de grafiek.

3.3 Deel 2: Global, Shared en Constant Memory

Voor het tweede deel werd een matrixvermenigvuldiging van 128×128 uitgevoerd om de prestaties van verschillende soorten GPU-geheugen te testen.



Figuur 5: Executietijd van een 128×128 matrixvermenigvuldiging afhankelijk van het gebruikte geheugentype.

- **Shared Memory (0.0109 ms):** Dit geheugentype levert met ruime voorsprong de beste prestaties. Omdat shared memory zich *on-device* en per *block* bevindt, kenmerkt het zich door een extreem lage access latency. Door de data strategisch in shared memory te laden en te verwerken, vermijden we herhaaldelijke en dure off-chip geheugentoegangen.

- **Global Memory (0.0153 ms):** Dit is de standaardoptie (*baseline*). Het is trager dan shared memory omdat global memory zich *off-device* bevindt en dus een aanzienlijk hogere latency heeft. Threads moeten voor elke bewerking data over de geheugenbus ophalen.
- **Constant Memory (0.3613 ms):** Constant memory blijkt in dit specifieke scenario verrassend traag. Hoewel dit *off-device* geheugen door caching is geoptimaliseerd voor leesacties met een lage latency, geldt dit snelheidsvoordeel alléén als alle threads in een warp tegelijkertijd *hetzelfde* geheugenadres opvragen (een *broadcast*). Omdat bij matrixvermenigvuldiging elke thread binnen de warp een unieke iteratie uitvoert en daardoor verschillende adressen uit de `c_B` array tegelijk opvraagt, forceert de hardware een *serialisatie* van deze verzoeken. Deze afhandeling één-voor-één veroorzaakt waarschijnlijk de extreme overhead.

4 Bijlage: C++ Code

```cpp
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#include <iostream>
#include <vector>
#include <iomanip>
#include <cmath>

const int NUM_ITERATIONS = 100;
const int BLOCK_SIZE = 16; // 16x16 = 256 threads per block

// Constante grootte voor Deel 2 (limiet is 64KB voor constant memory).
// 128 * 128 * sizeof(float) = 65536 bytes (64 KB)
const int N_MATMUL = 128;
__constant__ float c_B[N_MATMUL * N_MATMUL];

// ============================================================
// CPU VERIFICATIE FUNCTIES
// ============================================================
bool verifyMatrix(const std::vector<float>& A, const std::vector<float>& B, int N,
    float tolerance = 1e-4) {
    for (int i = 0; i < N * N; ++i) {
        if (std::abs(A[i] - B[i]) > tolerance) {
            return false;
        }
    }
    return true;
}

void cpuTranspose(const std::vector<float>& A, std::vector<float>& B, int N) {
    for (int row = 0; row < N; ++row) {
        for (int col = 0; col < N; ++col) {
            B[col * N + row] = A[row * N + col];
        }
    }
}

void cpuMatMul(const std::vector<float>& A, const std::vector<float>& B, std::vector<
    float>& C, int N) {
    for (int row = 0; row < N; ++row) {
        for (int col = 0; col < N; ++col) {
            float sum = 0.0f;
            for (int k = 0; k < N; ++k) {
                sum += A[row * N + k] * B[k * N + col];
            }
            C[row * N + col] = sum;
        }
    }
}

// ============================================================
// DEEL 1: COPY EN TRANSPOSE KERNELS
// ============================================================

__global__ void copy_kernel(const float* A, float* B, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (col < N && row < N) {
        int index = row * N + col; // 1D array indexing
```

```
58          B[index] = A[index];        // Coalesced read & coalesced write
59      }
60  }
61
62  __global__ void transpose_kernel(const float* A, float* B, int N) {
63      int col = blockIdx.x * blockDim.x + threadIdx.x;
64      int row = blockIdx.y * blockDim.y + threadIdx.y;
65
66      if (col < N && row < N) {
67          // A leest opeenvolgend (coalesced), maar B schrijft met sprongen (strided)
68          B[col * N + row] = A[row * N + col];
69      }
70  }
71
72  // ============================================================
73  // DEEL 2: MATMUL KERNELS
74  // ============================================================
75
76  __global__ void matmul_global(const float* A, const float* B, float* C, int N) {
77      int col = blockIdx.x * blockDim.x + threadIdx.x;
78      int row = blockIdx.y * blockDim.y + threadIdx.y;
79
80      if (col < N && row < N) {
81          float sum = 0.0f;
82          for (int k = 0; k < N; ++k) {
83              sum += A[row * N + k] * B[k * N + col];
84          }
85          C[row * N + col] = sum;
86      }
87  }
88
89  __global__ void matmul_shared(const float* A, const float* B, float* C, int N) {
90      // Statisch toegewezen shared memory per block
91      __shared__ float sA[BLOCK_SIZE][BLOCK_SIZE];
92      __shared__ float sB[BLOCK_SIZE][BLOCK_SIZE];
93
94      int tx = threadIdx.x; int ty = threadIdx.y;
95      int col = blockIdx.x * BLOCK_SIZE + tx;
96      int row = blockIdx.y * BLOCK_SIZE + ty;
97
98      float sum = 0.0f;
99
100     // Loop over de blokken
101     int numBlocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
102     for (int m = 0; m < numBlocks; ++m) {
103         // Kopieer naar shared memory (coalesced reads)
104         if (row < N && m * BLOCK_SIZE + tx < N)
105             sA[ty][tx] = A[row * N + m * BLOCK_SIZE + tx];
106         else
107             sA[ty][tx] = 0.0f;
108
109         if (col < N && m * BLOCK_SIZE + ty < N)
110             sB[ty][tx] = B[(m * BLOCK_SIZE + ty) * N + col];
111         else
112             sB[ty][tx] = 0.0f;
113
114         __syncthreads(); // Wacht tot het hele blok geladen is
115
116         // Bereken sub-matrix
117         for (int k = 0; k < BLOCK_SIZE; ++k) {
118             sum += sA[ty][k] * sB[k][tx];
```

```
119         }
120         __syncthreads(); // Zorg dat berekeningen klaar zijn voor nieuwe iteratie
121     }
122 
123     if (row < N && col < N) {
124         C[row * N + col] = sum;
125     }
126 }
127 
128 __global__ void matmul_constant(const float* A, float* C, int N) {
129     int col = blockIdx.x * blockDim.x + threadIdx.x;
130     int row = blockIdx.y * blockDim.y + threadIdx.y;
131 
132     if (col < N && row < N) {
133         float sum = 0.0f;
134         for (int k = 0; k < N; ++k) {
135             // Maakt gebruik van de gecachte constant memory voor B
136             sum += A[row * N + k] * c_B[k * N + col];
137         }
138         C[row * N + col] = sum;
139     }
140 }
141 
142 // ============================================================
143 // HOST CODE (CPU)
144 // ============================================================
145 
146 int main() {
147     cudaEvent_t start, stop;
148     cudaEventCreate(&start);
149     cudaEventCreate(&stop);
150 
151     // --- DEEL 1 SETUP ---
152     int N_p1 = 4096; // Grote matrix om bandbreedte goed te zien
153     size_t size_p1 = N_p1 * N_p1 * sizeof(float);
154     std::vector<float> h_A(N_p1 * N_p1, 1.0f);
155     std::vector<float> h_B(N_p1 * N_p1, 0.0f);
156 
157     // Willekeurige data
158     for (int i = 0; i < N_p1 * N_p1; ++i) h_A[i] = static_cast<float>(rand() % 100);
159 
160     float *d_A, *d_B;
161     cudaMalloc(&d_A, size_p1);
162     cudaMalloc(&d_B, size_p1);
163     cudaMemcpy(d_A, h_A.data(), size_p1, cudaMemcpyHostToDevice);
164 
165     dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
166     dim3 grid((N_p1 + BLOCK_SIZE - 1) / BLOCK_SIZE, (N_p1 + BLOCK_SIZE - 1) /
        BLOCK_SIZE);
167 
168     std::cout << "---␣DEEL␣1:␣Coalesced␣vs␣Strided␣Memory␣Access␣(" << N_p1 << "x" <<
         N_p1 << ")␣---\n";
169 
170     // Formule: Totale bytes gelezen en geschreven (2 arrays van grootte size_p1)
171     double total_bytes_GB = (2.0 * size_p1) / (1024 * 1024 * 1024);
172 
173     // 1. Copy Kernel
174     float time_copy = 0.0f;
175     for (int i = 0; i < NUM_ITERATIONS; ++i) {
176         cudaEventRecord(start);
177         copy_kernel<<<grid, threads>>>(d_A, d_B, N_p1);
```

```
178         cudaEventRecord(stop);
179         cudaEventSynchronize(stop);
180         float ms = 0; cudaEventElapsedTime(&ms, start, stop);
181         time_copy += ms;
182     }
183     time_copy /= NUM_ITERATIONS;
184     double bw_copy = total_bytes_GB / (time_copy / 1000.0);
185     std::cout << "Copy␣Kernel␣Tijd:␣␣␣␣␣␣" << time_copy << "␣ms␣\t␣Bandbreedte:␣" <<
            bw_copy << "␣GB/s\n";
186
187     // 2. Transpose Kernel
188     float time_transpose = 0.0f;
189     for (int i = 0; i < NUM_ITERATIONS; ++i) {
190         cudaEventRecord(start);
191         transpose_kernel<<<grid, threads>>>(d_A, d_B, N_p1);
192         cudaEventRecord(stop);
193         cudaEventSynchronize(stop);
194         float ms = 0; cudaEventElapsedTime(&ms, start, stop);
195         time_transpose += ms;
196     }
197     time_transpose /= NUM_ITERATIONS;
198     double bw_transpose = total_bytes_GB / (time_transpose / 1000.0);
199     std::cout << "Transpose␣Kernel␣Tijd:␣" << time_transpose << "␣ms␣\t␣Bandbreedte:␣
            " << bw_transpose << "␣GB/s\n";
200
201     cudaFree(d_A); cudaFree(d_B);
202
203     // --- DEEL 2 SETUP ---
204     int N_p2 = N_MATMUL; // 128x128 wegens limitatie constant memory
205     size_t size_p2 = N_p2 * N_p2 * sizeof(float);
206     std::vector<float> h_A2(N_p2 * N_p2, 2.0f);
207     std::vector<float> h_B2(N_p2 * N_p2, 3.0f);
208     std::vector<float> h_C2(N_p2 * N_p2, 0.0f);
209     std::vector<float> cpu_C2(N_p2 * N_p2, 0.0f);
210
211
212     std::cout << "---␣DEEL␣1:␣Schalen␣van␣de␣Matrixgrootte␣(N)␣---\n";
213     std::cout << "N␣|␣BW_Copy␣(GB/s)␣|␣BW_Transpose␣(GB/s)\n";
214
215     // kleine matrices
216     for (int N = 256; N <= 512; N *= 2) {
217         size_t size = N * N * sizeof(float);
218         double total_bytes_GB = (2.0 * size) / (1024 * 1024 * 1024);
219
220         float *d_A_scale, *d_B_scale;
221         cudaMalloc(&d_A_scale, size);
222         cudaMalloc(&d_B_scale, size);
223
224         dim3 threads_scale(BLOCK_SIZE, BLOCK_SIZE);
225         dim3 grid_scale((N + BLOCK_SIZE - 1) / BLOCK_SIZE, (N + BLOCK_SIZE - 1) /
                BLOCK_SIZE);
226
227         // Warmup voor deze N
228         copy_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
229         cudaDeviceSynchronize();
230
231         // 1. Copy Kernel meten
232         float time_copy_scale = 0.0f;
233         for (int i = 0; i < NUM_ITERATIONS; ++i) {
234             cudaEventRecord(start);
235             copy_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
```

```
236             cudaEventRecord(stop);
237             cudaEventSynchronize(stop);
238             float ms = 0; cudaEventElapsedTime(&ms, start, stop);
239             time_copy_scale += ms;
240         }
241         double current_bw_copy = total_bytes_GB / ((time_copy_scale / NUM_ITERATIONS)
                / 1000.0);
242
243         // 2. Transpose Kernel meten
244         float time_transpose_scale = 0.0f;
245         for (int i = 0; i < NUM_ITERATIONS; ++i) {
246             cudaEventRecord(start);
247             transpose_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
248             cudaEventRecord(stop);
249             cudaEventSynchronize(stop);
250             float ms = 0; cudaEventElapsedTime(&ms, start, stop);
251             time_transpose_scale += ms;
252         }
253         double current_bw_transpose = total_bytes_GB / ((time_transpose_scale /
                NUM_ITERATIONS) / 1000.0);
254
255         std::cout << N << "␣|␣" << current_bw_copy << "␣|␣" << current_bw_transpose
                << "\n";
256
257         cudaFree(d_A_scale);
258         cudaFree(d_B_scale);
259     }
260
261     // grotere matrices (veelvouden van 1024)
262     for (int N = 1024; N <= 24576; N += 1024) {
263         size_t size = N * N * sizeof(float);
264         double total_bytes_GB = (2.0 * size) / (1024 * 1024 * 1024);
265
266         float *d_A_scale, *d_B_scale;
267         cudaMalloc(&d_A_scale, size);
268         cudaMalloc(&d_B_scale, size);
269
270         dim3 threads_scale(BLOCK_SIZE, BLOCK_SIZE);
271         dim3 grid_scale((N + BLOCK_SIZE - 1) / BLOCK_SIZE, (N + BLOCK_SIZE - 1) /
                BLOCK_SIZE);
272
273         // Warmup voor deze N
274         copy_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
275         cudaDeviceSynchronize();
276
277         // 1. Copy Kernel meten
278         float time_copy_scale = 0.0f;
279         for (int i = 0; i < NUM_ITERATIONS; ++i) {
280             cudaEventRecord(start);
281             copy_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
282             cudaEventRecord(stop);
283             cudaEventSynchronize(stop);
284             float ms = 0; cudaEventElapsedTime(&ms, start, stop);
285             time_copy_scale += ms;
286         }
287         double current_bw_copy = total_bytes_GB / ((time_copy_scale / NUM_ITERATIONS)
                / 1000.0);
288
289         // 2. Transpose Kernel meten
290         float time_transpose_scale = 0.0f;
291         for (int i = 0; i < NUM_ITERATIONS; ++i) {
```

```
292             cudaEventRecord(start);
293             transpose_kernel<<<grid_scale, threads_scale>>>(d_A_scale, d_B_scale, N);
294             cudaEventRecord(stop);
295             cudaEventSynchronize(stop);
296             float ms = 0; cudaEventElapsedTime(&ms, start, stop);
297             time_transpose_scale += ms;
298         }
299         double current_bw_transpose = total_bytes_GB / ((time_transpose_scale /
                NUM_ITERATIONS) / 1000.0);
300
301         std::cout << N << "␣|␣" << current_bw_copy << "␣|␣" << current_bw_transpose
                << "\n";
302
303         cudaFree(d_A_scale);
304         cudaFree(d_B_scale);
305     }
306
307     std::cout << "\n---␣CPU␣Verificatie␣Deel␣2␣(MatMul)␣bezig...␣---\n";
308     cpuMatMul(h_A2, h_B2, cpu_C2, N_p2); // Voor correctheidscheck
309
310     float *d_A2, *d_B2, *d_C2;
311     cudaMalloc(&d_A2, size_p2);
312     cudaMalloc(&d_B2, size_p2);
313     cudaMalloc(&d_C2, size_p2);
314
315     cudaMemcpy(d_A2, h_A2.data(), size_p2, cudaMemcpyHostToDevice);
316     cudaMemcpy(d_B2, h_B2.data(), size_p2, cudaMemcpyHostToDevice);
317     // For constant memory, copy to the symbol
318     cudaMemcpyToSymbol(c_B, h_B2.data(), size_p2);
319
320     dim3 grid2((N_p2 + BLOCK_SIZE - 1) / BLOCK_SIZE, (N_p2 + BLOCK_SIZE - 1) /
            BLOCK_SIZE);
321
322     std::cout << "\n---␣DEEL␣2:␣Geheugentypes␣voor␣MatMul␣(" << N_p2 << "x" << N_p2
            << ")␣---\n";
323
324     // 1. Global Memory
325     float time_global = 0.0f;
326     for (int i = 0; i < NUM_ITERATIONS; ++i) {
327         cudaMemset(d_C2, 0, size_p2); // Reset C
328         cudaEventRecord(start);
329         matmul_global<<<grid2, threads>>>(d_A2, d_B2, d_C2, N_p2);
330         cudaEventRecord(stop);
331         cudaEventSynchronize(stop);
332         float ms = 0; cudaEventElapsedTime(&ms, start, stop);
333         time_global += ms;
334     }
335     cudaMemcpy(h_C2.data(), d_C2, size_p2, cudaMemcpyDeviceToHost);
336     std::cout << "Global␣Memory␣Tijd:␣␣␣" << (time_global / NUM_ITERATIONS) << "␣ms␣\
            t␣(Correct:␣" << (verifyMatrix(h_C2, cpu_C2, N_p2) ? "Ja" : "Nee") << ")\n";
337
338     // 2. Shared Memory
339     float time_shared = 0.0f;
340     for (int i = 0; i < NUM_ITERATIONS; ++i) {
341         cudaMemset(d_C2, 0, size_p2); // Reset C
342         cudaEventRecord(start);
343         matmul_shared<<<grid2, threads>>>(d_A2, d_B2, d_C2, N_p2);
344         cudaEventRecord(stop);
345         cudaEventSynchronize(stop);
346         float ms = 0; cudaEventElapsedTime(&ms, start, stop);
347         time_shared += ms;
```

```
348     }
349     cudaMemcpy(h_C2.data(), d_C2, size_p2, cudaMemcpyDeviceToHost);
350     std::cout << "Shared␣Memory␣Tijd:␣␣␣" << (time_shared / NUM_ITERATIONS) << "␣ms␣\
            t␣(Correct:␣" << (verifyMatrix(h_C2, cpu_C2, N_p2) ? "Ja" : "Nee") << ")\n";
351
352     // 3. Constant Memory
353     float time_const = 0.0f;
354     for (int i = 0; i < NUM_ITERATIONS; ++i) {
355         cudaMemset(d_C2, 0, size_p2); // Reset C
356         cudaEventRecord(start);
357         matmul_constant<<<grid2, threads>>>(d_A2, d_C2, N_p2);
358         cudaEventRecord(stop);
359         cudaEventSynchronize(stop);
360         float ms = 0; cudaEventElapsedTime(&ms, start, stop);
361         time_const += ms;
362     }
363     cudaMemcpy(h_C2.data(), d_C2, size_p2, cudaMemcpyDeviceToHost);
364     std::cout << "Constant␣Memory␣Tijd:␣" << (time_const / NUM_ITERATIONS) << "␣ms␣\t
            ␣(Correct:␣" << (verifyMatrix(h_C2, cpu_C2, N_p2) ? "Ja" : "Nee") << ")\n";
365
366     cudaFree(d_A2); cudaFree(d_B2); cudaFree(d_C2);
367     cudaEventDestroy(start); cudaEventDestroy(stop);
368
369     return 0;
370 }
```

Listing 1: L4a.cu CUDA implementatie met kernels voor copy, transpose en matrixvermenigvuldiging